PRAUDIT

NDB WEB APPLICATION
PENETRATION TEST

29-04-2026

INTERNATIONAL FEDERATION OF RED CROSS AND RED CRESCENT SOCIETIES

TABLE OF CONTENTS

Goal of the test 3
Scope of the test 3
Test circumstances 3
Test limitations 3
Testing location 4
Duration of the test 4
Received data, information 4
Executive summary 4
Clause 6
Conducted tests 7
Detailed findings 8
1. Application user details are accessible via low-privileged accounts 8
2. Account email addresses can be enumerated 9
3. Weak password policy 11
4. Email address can be changed 13
False positive findings 15
Annexes 16
Web application and HTTP API testing 16
Risk rating methodology 23

GOAL OF THE TEST

The goal of the assessment is to test the security of The International Federation of Red Cross and Red Crescent Societies (hereinafter IFRC or Client) NDB web application. The penetration test imitates a real attack in the sense that we use the tools and techniques a real-world adversary would use. Our assessment reveals the flaws that can be exploited to gain access to sensitive information, to elevate privileges or gain system level access to the infrastructure running the application.

This resulting document gives a high-level security overview of the target application and servers as well as detailed write-ups, possible impact and recommended remediation steps for each finding.

SCOPE OF THE TEST

The scope of the test covers the gray-box application test of NDB web application located at https://databank.ifrc.org/ .

TEST CIRCUMSTANCES

We conducted the test from the Internet through our two pentest servers: 193.201.185.90 and 193.201.185.91.

TEST LIMITATIONS

The application uses Azure WAF, which is a strong security practice, however, it significantly limited the range of attack techniques we were able to perform during testing. On the other hand, this limitation also constrains potential attackers, making the assessment realistic as long as Azure WAF remains in use.

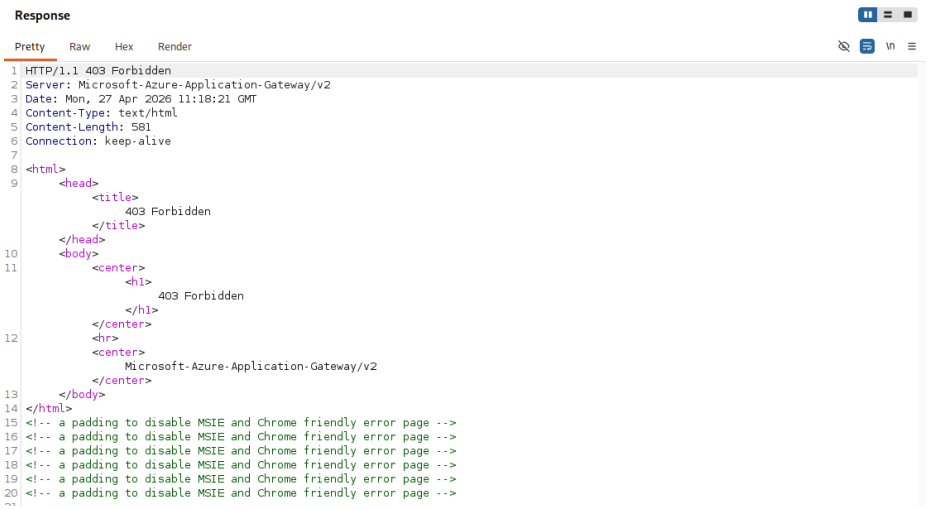

Figure 1 - Azure WAF

TESTING LOCATION

We conducted the tests remotely.

DURATION OF THE TEST

The tests were carried out between 13-04-2026 and 29-05-2026.

RECEIVED DATA, INFORMATION

Before the assessment we received the URL (https://databank.ifrc.org/) of the application and 6 users with 3 different user roles:

- pentest@praudit.hu: System Manager
- pentest1@praudit.hu: System Manager
- pentest2@praudit.hu: Admin
- pentest3@praudit.hu: Admin
- pentest5@praudit.hu: Focal Point
- pentest6@praudit.hu: Focal Point

EXECUTIVE SUMMARY

The assessment identified four security findings, with Medium risk ratings, affecting user privacy, account security, and access control within the application.

Two findings involve user enumeration and exposure of sensitive user information. A user with Focal Point rights can retrieve profile details for other users by manipulating the `user_ids` parameter in the `/api/users/profile-summary` endpoint. In addition, the login page discloses whether an email address belongs to a registered account. These issues could allow attackers or unauthorized users to identify valid application users, collect email addresses, review role assignments, and support targeted attacks such as phishing, credential stuffing, or brute-force attempts.

The assessment also found that the application allows commonly used predictable passwords, such as *Password1234!* and *Qwerty1234!*, despite enforcing general password complexity rules. While the risk is reduced by existing brute-force protection and account lockout controls, allowing well-known password patterns still increases the likelihood of compromise, particularly in targeted attacks or where users reuse passwords from previous breaches.

Finally, the application allows an administrator to modify a user's email address by sending the change directly to the backend, even though the user interface restricts edits to the email field. This indicates that the restriction is enforced only on the client side rather than properly validated by the server. The impact could be significant, as unauthorized email changes may enable password reset interception, redirection of account communications, or account takeover scenarios.

The tables below summarize our findings:

Table of findings

Ref.	Finding	Risk rating
1.	Application user details are accessible via low-privileged accounts	Moderate
2.	Account email addresses can be enumerated	Moderate
3.	Weak password policy	Moderate
4.	Email address can be changed	Moderate

CLAUSE

During the security tests, we took due care and diligence and acted upon the authorization of the Client. The assessment only covers the testing – carried out according to those written in the related methodology description – of the IT systems and security requirements featured in the offer and agreement.

We would like to draw the attention of the Client to the fact that the test is a kind of snapshot which is suitable for the static evaluation of the status of the IT system and does not track the changes made after project termination (system changes, recently discovered attack methods, vulnerabilities). During the tests, it is not our responsibility to examine the functional conformity of the systems under the scope.

It is our responsibility to draw the attention of the Client to the risks, the dangers they bear and the applicable best-practices for the security improvement of the networks, systems and applications of the Client. Client's responsibility covers the evaluation of the identified risks in the light of their business goals, costs and profit and the existing controls.

Furthermore, the management of the identified risks and their reduction to an acceptable level is also the responsibility of the Client. Throughout this process, Client can decide whether this goal is achieved through the suggestions made by us or any other means.

We would like to note that due to the time dedicated to the security testing being limited, it is impossible to identify every vulnerability. A malicious attacker (or group) not limited in time and tools and with adequate financial resources could succeed in bypassing the security measures.

CONDUCTED TESTS

The test steps performed during the assessment are summarized in the attached PRAUDIT_IFRC_NDB_pentest_20260430.xlsx document.

DETAILED FINDINGS

The detailed findings include the shortcomings identified during the test, their actual or potential impact, and our recommendations to correct them.

1. APPLICATION USER DETAILS ARE ACCESSIBLE VIA LOW-PRIVILEGED ACCOUNTS

Finding

A user with Focal Point rights can access detailed information about any application user, including their email address, last login, and roles, through the `/api/users/profile-summary` endpoint by changing the `user_ids` parameter.

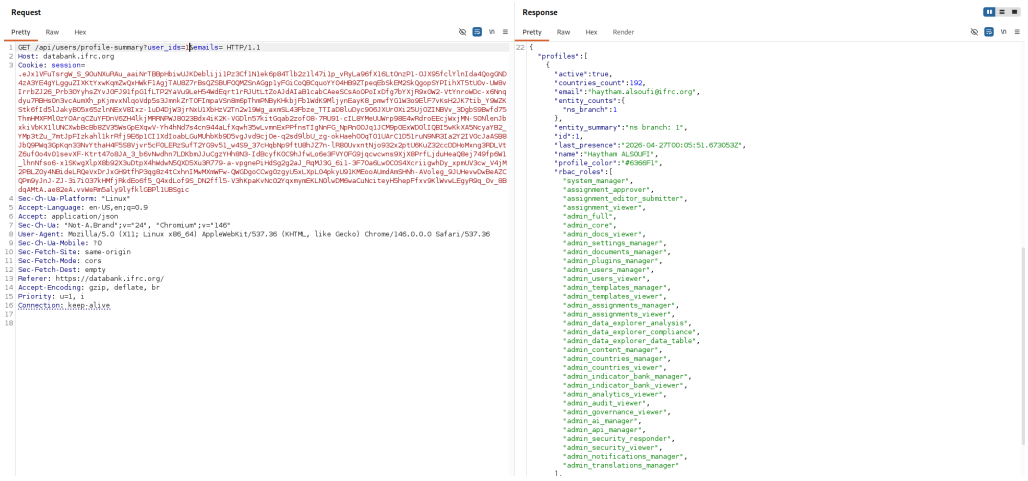

Figure 2 - User details

Impact

This issue may allow a user with Focal Point rights to retrieve sensitive profile information for other application users by modifying the `user_ids` parameter. Exposure of details such as email addresses, last login activity, and assigned roles could lead to privacy violations, user enumeration, and support further targeted attacks within the application.

Remediation

Restrict the `/api/users/profile-summary` endpoint so users can only access profile information they are explicitly authorized to view. The server should validate the requested `user_ids` against the requester's permissions and deny access to unauthorized records, regardless of client-side parameters.

CVSS4.0 Risk Rating:

Base value	5.3
Base vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N
Risk Rating	**Medium**

2. ACCOUNT EMAIL ADDRESSES CAN BE ENUMERATED

Finding

User email addresses can be enumerated through the application's login page. The backend response discloses whether the submitted email address is associated with a registered user account.

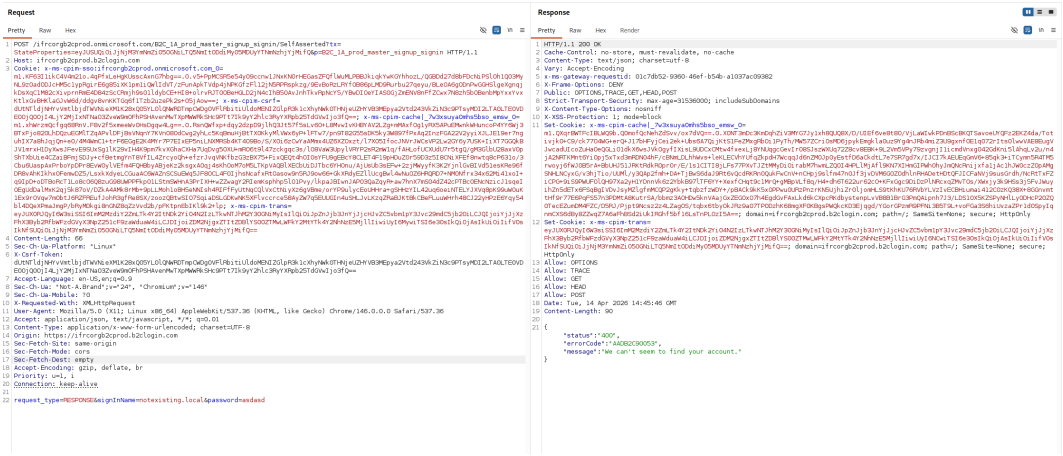

Figure 3 - Application response to a non-existing email address

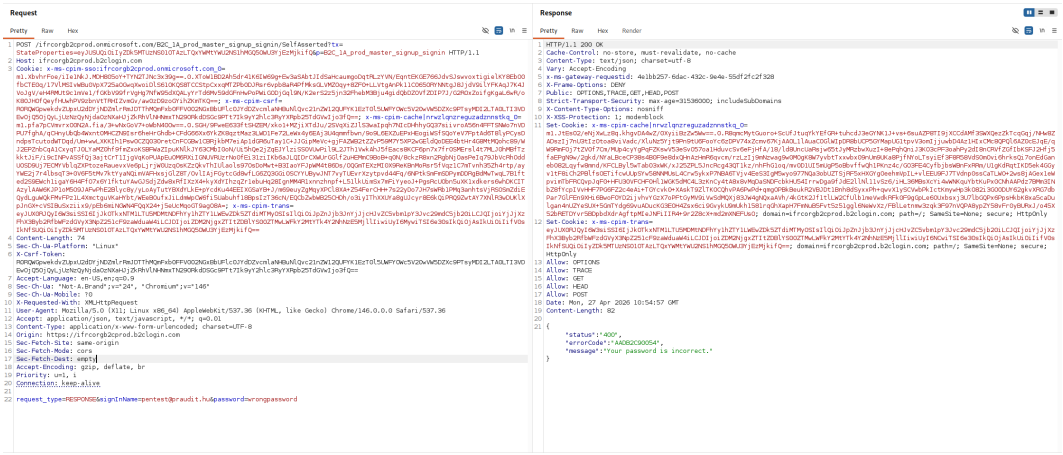

Figure 4 - Application response to a valid email address

Impact

This issue allows attackers to determine whether specific email addresses are registered in the application by observing differences in the login page responses. This can lead to user enumeration, privacy exposure, and may support targeted phishing, credential stuffing, or brute-force attacks against confirmed user accounts.

Remediation

Ensure the login page returns a generic response regardless of whether the submitted email address is registered.

CVSS4.0 Risk Rating:

Base value	6.3
Base vector	CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N
Risk Rating	**Medium**

3. WEAK PASSWORD POLICY

Finding

Although the application requires sufficiently strict password complexity, it does not block the use of commonly used sequential passwords. A user can use passwords such as "Password1234!" or "Qwerty1234!".

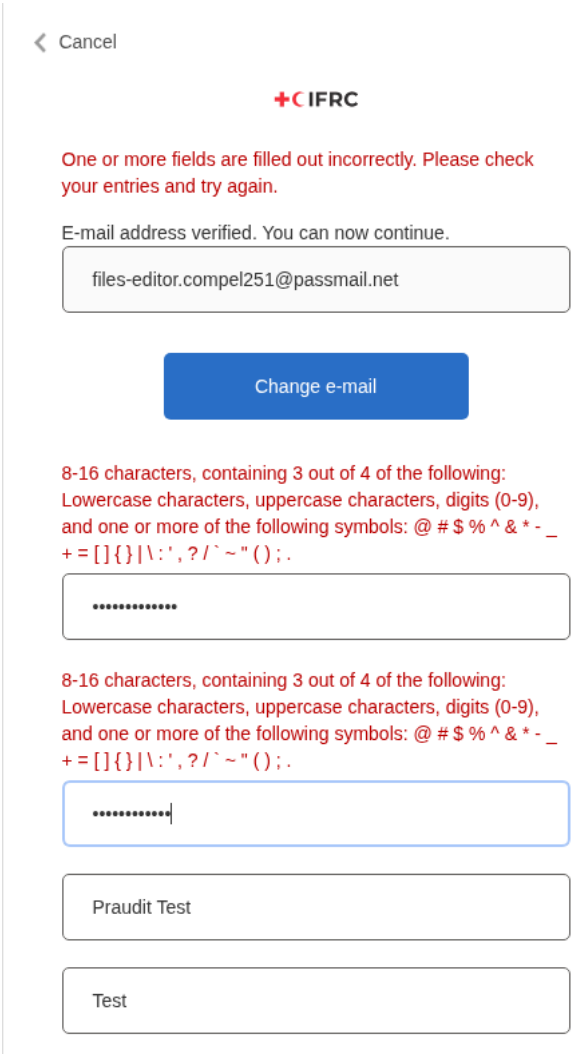

Figure 5 - Implemented password policy

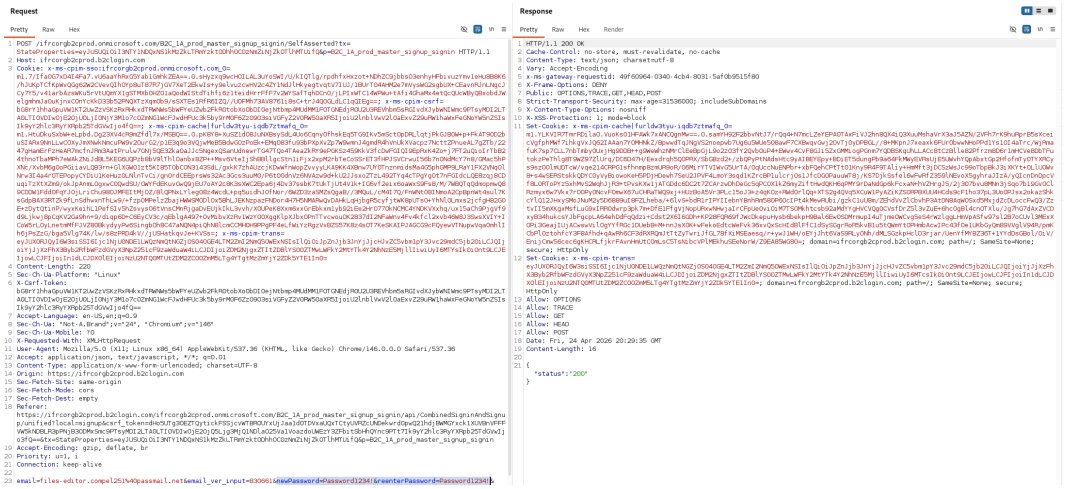

Figure 6 - Successful account creation with "Password1234!" password

It is important to note that the likelihood of exploiting this weakness is significantly reduced because the application implements brute-force protection and disables the account after a few incorrect login attempts.

Impact

Allowing commonly used sequential passwords may increase the risk of account compromise, especially if attackers attempt known password patterns or use credentials obtained from previous breaches. While brute-force protection reduces the likelihood of successful exploitation, users may still choose predictable passwords that are easier to guess in targeted attacks.

Remediation

We recommend blacklisting commonly used simple passwords (e.g., "Password1234!, "P@ssw0rd" etc.) as well as organization- or application-related terms and abbreviations and prohibiting the use of these words as parts of the passwords.

The following link lists the most common passwords in different lengths: https://github.com/berzerk0/Probable-Wordlists/tree/master/Real-Passwords .

And this website contains hashes of leaked passwords from almost all major data breaches: https://haveibeenpwned.com .

CVSS4.0 Risk Rating:

We want to emphasize that, due to the limitations of the CVSS 4.0 risk rating methodology, the finding received a Medium risk rating, however, in our professional opinion, this finding poses only a Low threat to the application.

Base value	6.3
Base vector	CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N
Risk Rating	**Medium**

4. EMAIL ADDRESS CAN BE CHANGED

Finding

An admin account can change the user's email address. The UI restricts modifications to the "Email" field; however, it can still be changed by sending a new email address to the application backend.

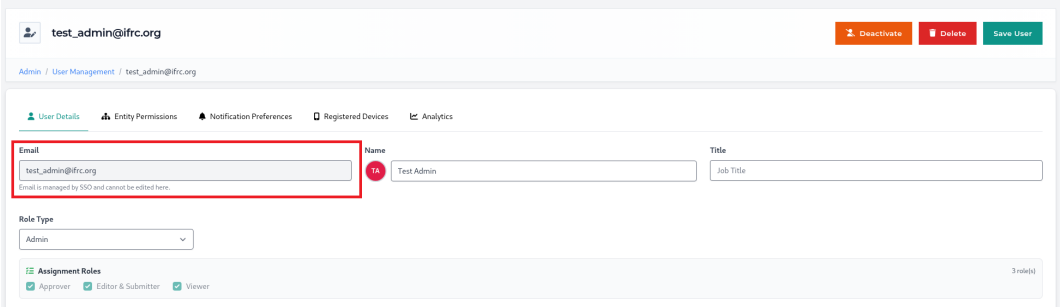

Figure 7 - UI restricts the modification of the email address

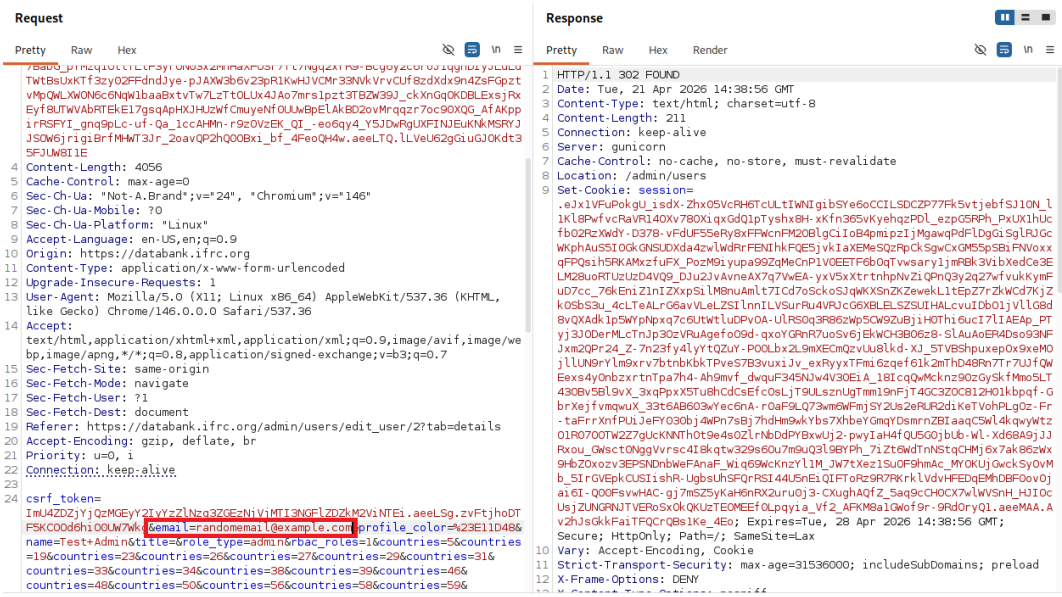

Figure 8 - Email address can be changed by sending a new value in the email POST parameter

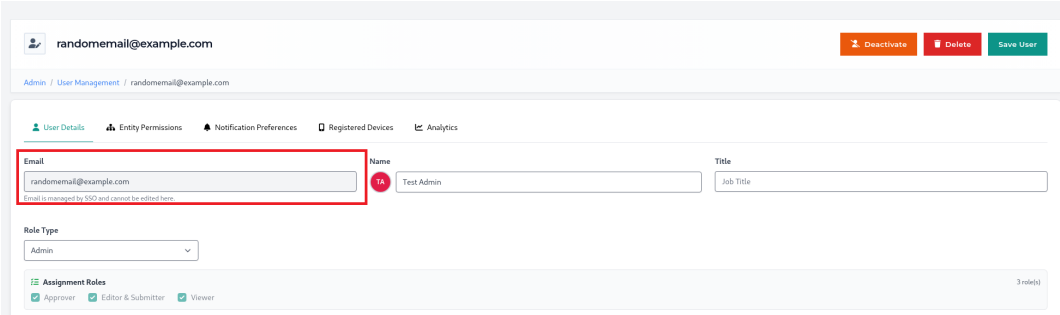

Figure 9 - Email address successfully changed

Impact

An administrator can modify a user's email address by bypassing the UI restriction and sending the change directly to the backend. This could allow account takeover, password reset interception, or unauthorized redirection of account-related communications.

Remediation

Enforce the email address restriction on the server side, not only in the UI. The backend should reject unauthorized email address changes, require appropriate role-based permission checks, and ideally log and notify users of any email change attempts.

CVSS4.0 Risk Rating:

Base value	5.1
Base vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:H/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N
Risk Rating	**Medium**

FALSE POSITIVE FINDINGS

Our automated vulnerability scans using Burp Suite Pro have identified the following issues. Manual checks determined that these are false positives.

Finding	Reason
Client-side desync	Burp Suite Pro reported a possible client-side desync vulnerability on the `/forms/assignment/1641` endpoint. Manual inspection revealed that this was a false positive finding, the vulnerability does not affect the server. (And even if it did, typical exploitation involves Cross-Site Scripting, which affects only browser-based applications).

ANNEXES

WEB APPLICATION AND HTTP API TESTING

Web application testing in itself is a significant and essential part of a security test. This is because nearly every organization has something web-related: a publicly accessible web page, some applications in the internal network with web interface, etc. We are discussing web application testing in its own chapter because of the wide range of technologies used on a web site leading to the many different attack vectors and testing methods making this topic a rather large one.

Our web application testing methodology is largely based on the industry standard ***OWASP (Open Web Application Security Project)***. The OWASP suggestions for application development and maintenance are widely accepted and used by standard like the ***Payment Card Industry Data Security Standard (PCI-DSS)***, which is a requirement for any e-commerce organization that stores, handles or transmits credit card information.

During the tests, we use both automatic and manual methods:

- ***Automatic tests:*** In this phase we use different free and commercial tools (***w3af, skipfish, Burp Suite, nikto***), that help us identify the possible threats. Using automatic tools can dramatically decrease the time the auditor has to spend on a web application, but automatic analysis can never replace thorough manual testing.
- ***Manual tests***: In order to verify the vulnerabilities found by the automatic tools, and to eliminate the false positive, we test the application manually as well. Also, automatic tools can find only typical threats, so a manual overview is always required for thorough analysis.

At the test steps corresponding to the points of OWASP Web Security Testing Guide (WSTG), we indicate the identification of the associated step.

INFORMATION GATHERING

In order to properly test a web application, we have to understand its functionality and layout. This is done by simply browsing the web page. During this, we get a clear view of the web application's components, its menu structure, and the applied technologies. Usually after this phase, we get an understanding about the possible attack vectors. The following questions are answered after this phase:

- What is the menu structure and what is the functionality behind each menu? (*WSTG-INFO-07, WSTG-INFO-08, WSTG-INFO-10*)
- What are the main components (e.g. authentication, messaging, etc.)? (*WSTG-INFO-10*)
- Is the application custom-made, or based on a CMS (e.g. Drupal, Joomla, etc.)? (*WSTG-INFO-08*)
- What technologies are used (e.g. PHP, ASP.NET, AJAX, JavaScript frameworks, etc.)? (*WSTG-INFO-08, WSTG-INFO-09*)
- How does a user interact with the application (e.g., traditional multi-page application, single page application, API-based interaction)? (*WSTG-INFO-06*)

Besides the above mentioned, the testing of the used webserver's type, version (*WSTG-INFO-02*) and also the information (*WSTG-INFO-03*) found in the default webserver files (e.g. default help

option, test pages, etc.), configurations (e.g. .htaccess files, robots.txt, etc.) and metadata (*WSTG-INFO-05*) found in other accessible files (e.g. images, documents) can be useful during an attack.

It does not matter how secure a web application is, when another less secure application is found on the same server, because the compromise of the less secure one can cause the compromise of the more secure one. For this, we test if components not associated with the target can be found on the same server (*WSTG-INFO-04*).

Based on the unique identifications (e.g. names of components, unique cookies, etc.) found in the application, we try to gather public information about the application using online search tools and public source code storage (e.g. GitHub) (*WSTG-INFO-01*).

VULNERABILITY TESTS

During our analysis, we primarily test vulnerabilities listed in the ***OWASP*** (***Open Web Application Security Project***) ***Top Ten Project's 2025 release:***

- ***A01 - Broken Access Control:*** This vulnerability combines insecure direct object access, which means that certain internal information (such as configuration files, database passwords) is accessible via the web and is easily accessible to an attacker; and the problem that some web applications only authenticate, authorize the user before the user interface is displayed, and simply do not display links, menu items that the user has no rights to. However, if an attacker knows the address at which you can access sensitive data or functions, this "protection" is not enough.
- ***A02 - Security Misconfiguration:*** A well-written, secure web application can also be vulnerable if the web server or some external libraries, plugins or other components are not properly configured. (e.g. enabled unnecessary services, default credentials, missing security settings).
- ***A03 - Software Supply Chain Failures:*** This vulnerability arises when the software development lifecycle - from building, dependency management, to distribution - is compromised by third-party components, outdated libraries, malicious packages, or insecure CI/CD pipelines, allowing attackers to inject malware or exploit known vulnerabilities in the supply chain.
- ***A04 - Cryptographic Failures:*** this category contains issues regarding encryption of stored, processed and sent data, such as plaintext communication, outdated/vulnerable SSL protocols and algorithms, missing HSTS header, weak hashing algorithms etc.
- ***A05 - Injection:*** This kind of vulnerability occurs when an application sends untrusted data to an interpreter. (E.g. SQL, LDAP, XML interpreter) If a web application contains a vulnerability of this kind, an attacker could have the opportunity to access confidential data or even run a code on the vulnerable system. This type includes cross-site scripting vulnerabilities as well.
- ***A06 - Insecure Design:*** some vulnerabilities: Not all vulnerabilities are the result of incorrect implementation; this category includes vulnerabilities caused by the lack of "Secure Design". For example: error messages containing sensitive information, insecure password reset, lack of protection against bots.
- ***A07 - Authentication Failures:*** The authentication and authorization of the users and session management are complex tasks, and as it is, its realization is often flawed. The flaws found in these subsystems make it possible for an attacker to act in the name of another user and access data and functions, which they should not have the privilege for.
- ***A08 - Software or Data Integrity Failures:*** Insufficient validation of the authenticity and integrity of the software components or data used by the application poses a serious risk. This category also includes insecure deserialization: if a web application does not properly process serialized content (e.g. XML, JSON, Java objects) from a user, an attacker

can create arbitrary objects, which in the worst case could even lead to remote code execution on the server.

- ***A09 - Security Logging and Alerting Failures:*** By inspecting the logs of the application and the operating system that serves it, typical attacks can be detected even by automatic methods. Managing alerts for such events can prevent ongoing attacks, so adequate logging settings are important. We only test these if source code analysis and/or hardening assessment is covered by the testing scope.
- ***A10 - Mishandling of Exceptional Conditions:*** This vulnerability arises when applications fail to properly handle errors, exceptions, or unexpected inputs, often defaulting to insecure states (e.g., "fail open" access). Attackers can exploit this to cause denial of service, leak sensitive information through error messages, bypass security controls, or trigger unintended behaviors during crashes or edge cases.

VULNERABILITIES RESULTING FROM MISCONFIGURATION

The settings of the server environment, webserver and frameworks are also as important security-wise as the web application itself. An inadequate configuration may lead to the leak of sensitive data or even compromise of the whole application (*WSTG-INFO-07*, *WSTG-INFO-08, WSTG-INFO-09, WSTG-INFO-10*). This phase also includes testing for missing or improperly configured HTTP Strict Transport Security (HSTS), insecure Rich Internet Application (RIA) cross-domain policies, improper file and directory permissions, subdomain takeover conditions, and exposed or misconfigured cloud storage resources that may allow unauthorized access to sensitive data or application components (*WSTG-CONF-07 to WSTG-CONF-11*).

Mapping the usable HTTP methods (*WSTG-INFO-06, WSTG-CONF-06*) is important for two reasons:

- Filtering methods that mean security risks (e.g. TRACE)
- The presence of certain methods may give away the accessible functions (e.g. PROPFIND method implies WebDav service)

An accessible administration web interface, a service port providing management functions left open and the temporary, or backup save files accessible in the webroot often provide great starting points for attacking an application, thus exploring them is a crucial step of our testing. These tests correspond with the points from *WSTG-CONF-01* to *WSTG-CONF-05* in the OWASP WSTG.

This phase covers the API8:2023 - Security Misconfiguration part of OWASP Top 10 API Security Risks – 2023 in case of HTTP API testing.

USER MANAGEMENT

Most web applications give opportunity for logging in, using functions linked to authentication and authorization. We must understand what privileges each user type should have and test if they are implemented according to the specifications (*WSTG-IDNT-01*). During the testing of the user registration process, we analyze, who, on what conditions can add users to the system and if this process can be bypassed in any way (*WSTG-IDNT-02, WSTG-IDNT-03*). Testing if an application has forced password policy and how well it satisfies the security requirements (*WSTG-ATHN-07*) is an important step of the analysis of the registration process.

As a part of the registration process, often there is possibility – in certain cases it is mandatory – to determine security questions, which are used for restoring passwords by the application. If it is implemented inappropriately, an attacker may take advantage of this function to change or obtain

the password of a legitimate user (*WSTG-ATHN-08*). The password restoring function may mean a security risk in itself, for instance, in the cases of tokens not suitably generated (e.g. created by using an insecure random number generator giving predictable matches), passwords sent unencrypted via e-mail (*WSTG-ATHN-09*).

During a brute-force attack, it is a great help if the usernames can be guessed in some ways (e.g. usernames following the "first name.last name" pattern are easily manufactured from the employee list of an organization, or – especially in the case of sites with considerable traffic – attempts with usernames made up from the most frequent name may succeed). Further simplifies the attacker's job, if the application responds differently to login attempts using existing and non-existing usernames (*WSTG-IDNT-04, WSTG-IDNT-05*).

AUTHENTICATION

We analyze the authentication layer of a web application from different angles. Firstly, it is important for the sensitive information (e.g. passwords) used for logging in, to move safely onto the network during the complete process (*WSTG-ATHN-01, WSTG-CRYP-03*). Secondly, they must never be stored unencrypted (*WSTG-ATHN-05, WSTG-ATHN-06*). A problem occurring in the case of content management systems and other frameworks, is that components linked to certain authentications get into the live operation without the default password being changed. This may often lead to the compromise of the whole application and even the webserver (*WSTG-ATHN-02*).

For stopping brute-force attacks that take advantage of the lack of the users' security awareness and are aimed at obtaining weak user account passwords, it is essential for a web application to use some kind of mechanism for detecting attempts of this nature and for temporarily blocking the account (*WSTG-ATHN-03*).

The security actions mentioned above may completely lose effect if the authentication process is inappropriately implemented and is bypassable. Because of the diversity of the applied authentication solutions, the possible errors cannot be generalized but here should be mentioned, for instance, the accessibility – without login - of the components theoretically linked to authentication (e.g. if the attacker knows the access route, URL) and also the not suitable implementation of the different SSO solutions (*WSTG-ATHN-04, WSTG-ATHN-10*).

This phase covers the API2:2023 - Broken Authentication of OWASP Top 10 API Security Risks – 2023 in case of HTTP API testing.

AUTHORIZATION

Testing the interoperability between the individual users or user groups - horizontal (e.g. a user can access another user's data) and vertical (e.g. the basic user can access administrative functions) privilege extensions – is done through studying the authorization process and exploring loopholes (*WSTG-ATHZ-02, WSTG-ATHZ-03*). During this, we use the information on user groups, roles collected under User management.

A frequently occurring flaw with web applications is, files (even outside the webroot!) are accessible knowing the direct URL or taking advantage of certain functions of the application (e.g. if the application shows the content of a file sent through unverified URL query parameter).This, in given cases, can lead to remote code execution, compromise of the webserver. Exploring flaws of this nature is done by analyzing queries launched, while using the application and by modifying the parameters (*WSTG-ATHZ-01, WSTG-ATHZ-04*).

This phase covers API1:2023 - Broken Object Level Authorization, API3:2023 - Broken Object Property Level Authorization and API5:2023 - Broken Function Level Authorization parts of OWASP Top 10 API Security Risks – 2023 in case of HTTP API testing.

SESSION MANAGEMENT

The HTTP protocol cannot manage statuses, thus almost every modern web application solves the storage of the current status on application level by creating sessions. Generally, it means that, at the log in, the application creates a session on the server side, and the client sends the unique identification of this session to the server at every query. The process, from security angles, could contain multiple flaws. Testing these is a must during the security testing of a web application.

Nowadays, less frequently, but it occurs that the unique identification of the session is easily guessed (e.g. it generates from the username through the known algorithm). Testing this is done by analyzing the session identifications (*WSTG-SESS-01, WSTG-SESS-08*). Sometimes the identification cannot be guessed, however after logging in, the systems use the same identification – this gives opportunity for a so-called session fixation attack, during which, the attacker using social engineering methods, manipulates a user to log into the application with a session identification known by them (*WSTG-SESS-03*).

Often the session's unique identification appears in places, which in given cases, can be stored or be easily obtained (e.g. the session ID sent through GET parameter is stored in the browser history). We test this by analyzing requests aimed at the server (*WSTG-SESS-04*). A problem linked to this is, when the session ID is stored, sent in cookies, on which the appropriate security attributes are not set up (*WSTG-SESS-02*).

Ideally, a session is only valid for a given period and after exiting the application, it is destroyed (*WSTG-SESS-06, WSTG-SESS-07*).

The most frequently occurring security deficiency regarding session management is the so-called Cross-Site Request Forgery. There are effective methods against it, the lack of which can be easily determined from the queries sent by the application (WSTG-SESS-05).

We also test whether the session can be hijacked (e.g. by stealing or replaying session identifiers/cookies), which would allow an attacker to impersonate the victim user (WSTG-SESS-09).

This phase covers the API7:2023 - Server-Side Request Forgery part of OWASP Top 10 API Security Risks – 2023 in case of HTTP API testing.

TESTING INPUT DATA

The most frequent flaws regarding web application security are all due to the inadequate control of data coming from users. Because of this, data validation testing is a fundamental part of testing, focusing on how the application handles, filters and processes user-supplied input across all entry points. The most commonly known flaws of this nature are the XSS (*WSTG-INPV-01, WSTG-INPV-02, WSTG-CLNT-01, WSTG-CLNT-02*) and the SQL injection (*WSTG-INPV-05*) but depending on the applied technologies and protocols, there may be other, similar attacks:

- LDAP injection (*WSTG-INPV-06*)
- XML, XPath injection (*WSTG-INPV-07, WSTG-INPV-09*)
- SSI injection (*WSTG-INPV-08*)
- IMAP/SMTP injection (*WSTG-INPV-10*)
- Format string injection *(WSTG-INPV-13)*

Other important types of injection attacks are the code and command injection (*WSTG-INPV-11, WSTG-INPV-12*), , server-side template injection *(WSTG-INPV-18*), local and remote file embedding, as well as server-side request forgery (*WSTG-INPV-19*). All of these could also make remote code execution possible even on the webserver.

Less frequent problems are the deficiencies derived from the modification of HTTP queries, embedding them into other protocols or abusing protocol handling mechanisms, such as HTTP verb tampering, HTTP parameter pollution, HTTP response splitting or request smuggling, host header injection and improper handling of incoming HTTP requests *(WSTG-INPV-03, WSTG-INPV-04, WSTG-INPV-15, WSTG-INPV-16, WSTG-INPV-17)*.

XXE (XML External Entity) testing belongs here, as well. XML files may contain data from an external source (from another file, a service available on the network). If the application provides an opportunity to upload an XML file, exploiting an XXE deficiency may allow access to sensitive data or launch additional attacks on the internal network.

Deserialization attacks also belong to this phase. Deserialization attacks use the function of certain program languages that allow you to save objects to a file and to restore them later. In case of inadequate controls, it is possible to create a malicious serialized object and then load it with the application, which may result in arbitrary code execution or delayed exploitation scenarios when the stored data is later processed by the application.

This phase covers the API10:2023 - Unsafe Consumption of APIs part of OWASP Top 10 API Security Risks – 2023 in case of HTTP API testing.

ERROR MANAGEMENT

If a web application does not appropriately handle errors occurring during usage, it is possible that error messages show on the web interface indicating the application's structure and the applied technologies so it could be useful for an attacker. These usually appear in detail in the form of stack traces, error messages coming directly from the backend. Testing is done by analyzing error pages (*WSTG-ERRH-01, WSTG-ERRH-02*).

WEAK ENCRYPTION

If the SSL/TLS connection used by the HTTPS protocol is not set up adequately, it will not work correctly or will not work at all, allowing eavesdropping and tempering with the network traffic. Because of this, it is especially important to test the compliance of these configurations. This, we do by automatic screening of the information gathered from the SSL/TLS connection with the server (*WSTG-CRYP-01, WSTG-CRYP-03*). During this phase, weak or insecure encryption mechanisms used by the application are identified, such as weak cryptographic algorithms, insecure modes of operation, insufficient key lengths or improper randomness; however, these issues are not always directly testable through the web application itself, and in some cases can only be identified based on accessible configuration files or exposed implementation details (*WSTG-CRYP-04*).

In the case of data stored encrypted, there may also be flaws which allow extraction of unencrypted contents. The most common reason for this is the usage of non-standard, self-designed encryption algorithms or the implementation flaws in the general solutions, for instance, different padding oracle deficiencies (*WSTG-CRYP-02*).

LOGICAL FLAWS

The business logical planning and implementation flaws, which define the application's operation and mechanics are typically the ones that can only be found via manual tests, after understanding the operation of the application – the automatic tools are not much help with the exploration of these deficiencies. Because of the infinite number of tasks and solutions imaginable, these flaws cannot be clearly categorized, associated with test steps – the steps listed below, found in the OWASP Testing Guide only paint a general image of the possible test scenarios:

- Test business logic data validation (*WSTG-BUSL-01*)
- Test Ability to forge requests (*WSTG-BUSL-02*)
- Test integrity checks (*WSTG-BUSL-03*)
- Test for Process Timing (*WSTG-BUSL-04*)
- Test number of times a function can be used limits (*WSTG-BUSL-05*)
- Testing for the Circumvention of Work Flows (*WSTG-BUSL-06*)
- Test defenses against application mis-use (*WSTG-BUSL-07*)
- Test Upload of Unexpected File Types (*WSTG-BUSL-08*)
- Test Upload of Malicious Files (*WSTG-BUSL-09*)

Finding the logical flaws and the methods for taking advantage of them is possible through studying the application's structure and operation in detail and through understanding the processes taking place during the usage of the application. Throughout our tests, we strive for completeness, thus these tests are also vital parts of a web application security audit.

This phase covers the API6:2023 - Unrestricted Access to Sensitive Business Flows part of OWASP Top 10 API Security Risks – 2023 in case of HTTP API testing.

CLIENT-SIDE FLAWS

We also test other client-side injection attack methods (e.g. HTML, CSS injection) besides the previously discussed XSS flaw tests mostly regarding the client-side (*WSTG-INPV-01, WSTG-INPV-02, WSTG-CLNT-01, WSTG-CLNT-02*). Many times, the cause of these is the inappropriate input validation similar to XSS and the testing is also conducted similarly to XSS (*WSTG-CLNT-03, WSTG-CLNT-05, WSTG-CLNT-06*). We test the usage of the different, modern web technologies implemented in browsers as well (CORS - *WSTG-CLNT-07*, WebSockets - *WSTG-CLNT-10*, Web Messaging - *WSTG-CLNT-11*, Local Storage - *WSTG-CLNT-12*, Cross Site Script Inclusion - *WSTG-CLNT-13*).

The not validated URL redirect and the clickjacking attack forms belong to the frequent client-side security deficiencies. The former can be checked via testing the requests aimed at the server and the latter via testing the response headers found in the responses coming from the server (*WSTG-CLNT-04, WSTG-CLNT-09, WSTG-CLNT-08*).

EXPLOITATION

In this phase, we try to exploit the vulnerabilities found during the previous phases in order to verify the real threats and eliminate all false positive findings. We intensively utilize ***Metasploit Framework*** and ***Armitage*** as well as our own scripts during the exploitation phase. The methods used here are intrusive, meaning that they may cause unexpected behavior in the system (e.g., we create new users, new services, etc.). In the course of the tests, we exercise particular caution to eliminate the risk of causing any harm to the Client's system.

RISK RATING METHODOLOGY

We use FIRST's (FIRST.Org, Inc.) Common Vulnerability Scoring System version 4.0 (CVSS v4.0) as our risk rating methodology. The Common Vulnerability Scoring System (CVSS) is an open, industry-standard framework designed to communicate the characteristics and severity of software vulnerabilities. CVSS 4.0 is composed of four metric groups: Base, Threat, Environmental, and Supplemental. In the risk rating section of our reports, we apply only the Base metrics, formally referred to as "CVSS-B."

Below, we provide a brief overview of the scoring system based on the official CVSS v4.0 specification. This overview outlines the applied metric categories (Exploitability and Impact), includes a short description of each, and briefly enumerates their possible values. In addition, we provide a short discussion of other – so called subsequent – affected systems. A detailed explanation is available in the official specification document at https://www.first.org/cvss/v4.0/specification-document.

EXPLOITABILITY METRICS

The Exploitability metrics reflect the ease and technical means by which the vulnerability can be exploited.

Metric value	Description	Values (higher to lower severity)
Attack Vector (AV)	Reflects the context by which vulnerability exploitation is possible. This metric value will be larger the more remote (logically, and physically) an attacker can be in order to exploit the vulnerable system.	**Network:** Bound to the network stack, allowing remote exploitation without physical access (e.g., over the Internet) **Adjacent**: Bound to a protocol stack, limited to a logically adjacent topology (e.g., local IP subnet or WLAN) **Local**: Requires a local account or physical access to the target system (e.g. logged in user, phishing) **Physical**: Demands physical interaction with the hardware or target (e.g. USB device)

Metric value	Description	Values (higher to lower severity)
Attack Complexity (AC)	Captures measurable actions that must be taken by the attacker to actively evade or circumvent existing built-in security-enhancing conditions in order to obtain a working exploit.	**Low:** No target-specific circumvention needed, repeatable success. **High**: The successful attack depends on the evasion or circumvention of security-enhancing techniques
Attack Requirements (AT)	Evaluates if specific deployment or execution conditions of the vulnerable system must exist for the attack to succeed; if unmet, success may be occasional or impossible.	**None**: Vulnerable system exploitable under all or most conditions **Present**: Specific deployment conditions present that enable attack
Privileges Required (PR)	Measures the lowest privilege level an attacker needs on the vulnerable component to exploit it.	**None**: No authentication required (e.g., anonymous access). **Low**: Basic user-level access (e.g., standard account). **High**: Significant control over the vulnerable system, full access (e.g. Administrative/root privileges)
User Interaction (UI)	Captures the requirement for a human user, other than the attacker, to participate in the successful compromise of the vulnerable system.	**None**: No targeted user interaction required **Passive**: Limited interaction by the targeted user (e.g. XSS, CSRF, malware previously planted on system) **Active**: Specific, conscious interaction by targeted user

IMPACT METRICS (VULNERABLE/SUBSEQUENT SYSTEM)

The Impact metrics reflect the direct consequence of a successful exploit and represent the consequence to the "things that suffer the impact", which may include impact on the vulnerable system and/or the downstream impact on what is formally called the "subsequent system(s)".

Metric value	Description	Values (higher to lower severity)
Confidentiality (VC/SC)	How much and what kind of data an attacker can access post-exploitation on the vulnerable system.	**High**: Total loss of confidentiality (e.g., all data divulged) or disclosed information presents a direct, serious impact (e.g. private keys, administrator passwords) **Low**: Some restricted info disclosed, limited usefulness, no direct, serious loss **None**: No confidentiality loss.
Integrity (VI/SI)	How much and what kind of data an attacker can modify or delete post-exploitation on the system.	**High:** Total loss of integrity, or a complete loss of protection (e.g. the attacker can modify any file, or the affected files present direct, serious risk. **Low**: No control over the consequence of a modification, or the amount of modification is limited. **None**: No integrity loss.
Availability (VA/SA)	How much and what kind of service functionality an attacker can disrupt post-exploitation on the vulnerable system.	**High**: Total loss of availability or the loss of availability presents a direct, serious consequence to the system **Low**: Performance is reduced or there are interruptions in resource, but limited attack surface. **None**: No impact on availability.

SUBSEQUENT SYSTEM

Successful exploitation might affect other systems besides the vulnerable one. The methodology refers to these as subsequent systems, and their impact metrics contribute to the overall risk rating. While the vulnerable system is typically a software application, operating system, module, driver, etc. (or possibly a hardware device), the subsequent system could be any of those examples but also includes human safety.

If specific Impact metrics value is None for subsequent system, it means there is no specific loss within the Subsequent System, or all impact is constrained to the Vulnerable System.

CVSS V4.0 SCORING

A CVSS v4.0 score is represented as a vector string that concisely summarizes the selected metric values. The vector begins with the CVSS version identifier, followed by individual metric–value pairs separated by slashes.

For example: **CVSS:4.0/AV:N/AC:H/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N**

The vector is read from left to right. Each abbreviated metric (such as **AV** for Attack Vector or **VC** for Confidentiality Impact on the Vulnerable System) is followed by its assigned value. Together, these values describe how the vulnerability can be exploited and the resulting impact on the affected and subsequent systems.

For vulnerability scoring, the official CVSS v4.0 calculator can be used (https://www.first.org/cvss/calculator/4.0). For example, the previously mentioned vector results in a score of **6.3 (Medium).**

The qualitative severity ratings corresponding to CVSS scores are shown below.

CVSS Score	Rating
0.0	**None**
0.1 - 3.9	**Low**
4.0 - 6.9	**Medium**
7.0 - 8.9	**High**
9.0 - 10.0	**Critical**